

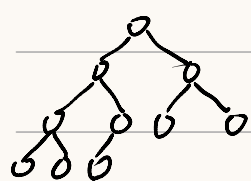
堆

维护一个数据集

- 1° 插入一个数
- 2° 求集中的最小值 (小顶堆)
- 3° 删除最小值
- 4° 删除任意元素
- 5° 修改任意元素

STL中的堆可直接使用

堆是一个完全二叉树



除最后一层, 其余所有层都是满的

最后一层严格从左到右排布

任一结点小于其子结点 (若有在)

完全二叉树

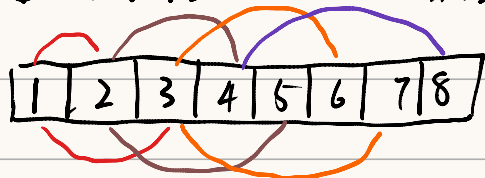
小顶堆

这样根结点就是最小值

堆的存储: 用一维数组存

1号为根结点

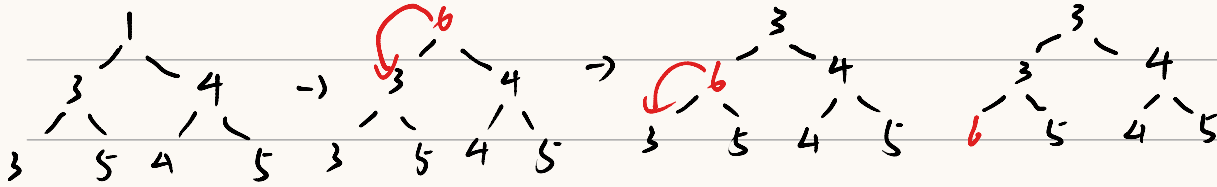
x 的左儿子: $2x$ 右儿子: $2x+1$



操作:

1° down(x) 向下调整 (大元素下坠)

头结点值变化时, 要调整头结点位置, 使整体仍为堆

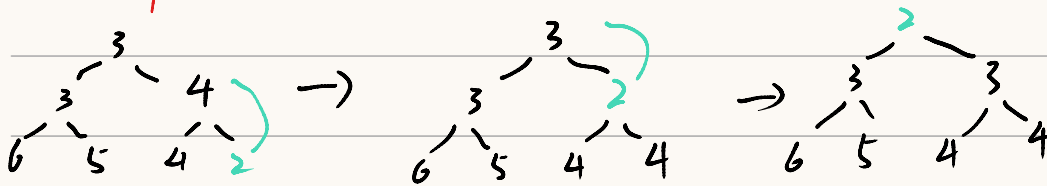


1° 头结点比两子结点都小 不动

2° 头结点比一个子结点大 二者交换

3° 头结点比两个子结点都大 与子结点中更小的那个交换

2° up(x) 向上调整 (小元素上移)



3° 插入: $\text{heap}[++\text{size}] = x$
 $\text{up}(\text{size})$

4° 求 min $\text{heap}[1]$

5° 删除 min 用最后一个元素覆盖堆顶, 再 down 数组尾删比头删快

$\text{heap}[1] = \text{heap}[\text{size}--]$

$\text{down}(1)$

6° 删任意元素 $\text{heap}[k] = \text{heap}[\text{size} - 1]$

$\text{down}(k)$ $\text{up}(k)$

1° $\text{heap}[k]$ 变大, 应该 down 2° 变小, 应该 up 3° 不变 不动
这里虽然 down , up 都写了, 但实际只会执行一个

7° 修改任意 $\text{heap}[k] = x$

$\text{down}(k)$ $\text{up}(k)$

补充: 从 1 开始原因: $0 \times 2 = 0$, 若从 0 开始, 需要特判

所以从 1 开始 0 可以存 size ★

线性建堆法:

对于一组数进行建堆 最终树高: $\log_2 N$

$\therefore \text{down}, \text{up}$ 时间复杂度: $O(\log_2 N)$

建法: ① 遍历数组, 依次把数插到堆尾, 再 up

$O(n \log n)$

② 线性建堆法:

一棵完全二叉树中叶子结点数量:

n_0 : 叶结点 n_1 : 只有 1 个子结点的结点 n_2 : 2 个子结点的结点

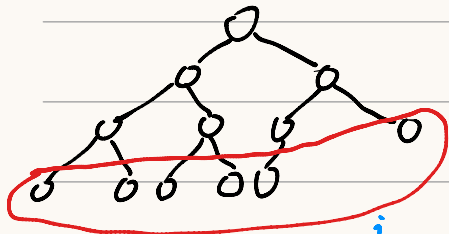
$$\left\{ \begin{array}{l} \text{点: } n_0 + n_1 + n_2 = n \\ \text{边: } n_1 + 2n_2 = n - 1 \quad \text{边数} = \text{点数} - 1 \end{array} \right\} \quad n_0 = n_2 + 1$$

$n_1 = 1 \text{ 或 } 0$ (完全二叉树性质)

① $n_1=0 \Rightarrow n_0=n_2+1$ 叶结点数量为 $\lfloor \frac{n}{2} \rfloor + 1$

② $n_1=1 \Rightarrow n_0=n_2+n_1$ 叶结点数量: $\frac{n}{2}$

综上: 完全二叉树中, 叶结点数量 $\geq \frac{n}{2}$



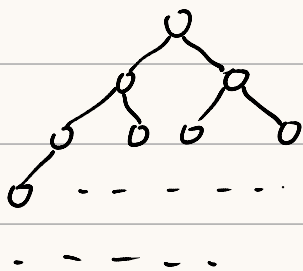
每个叶结点自成一个小元素堆, 不用调整

我们只用调整非叶结点

$i: 1 \sim \frac{n}{2}$ down(i)

从最后一个非叶结点开始 down

时间复杂度分析:



$\frac{n}{2}$
 $\frac{n}{4}$
 $\frac{n}{8}$
当前层结点数量

$$\sum_{i=1}^{\infty} \frac{n}{2^i} = n$$

$$\frac{\frac{1}{2}(1-(\frac{1}{2})^n)}{1-\frac{1}{2}} = 1$$

$\frac{n}{2}$ 为叶结点, 不动 $\frac{n}{4}$ 最多向下走 1 次 $\frac{n}{8}$ 最多 2 次 ...

$$S = \frac{n}{2} \times 0 + \frac{n}{4} \times 1 + \frac{n}{8} \times 2 + \frac{n}{16} \times 3 \dots$$

$$\frac{1}{2}S = \frac{n}{8} \times 1 + \frac{n}{16} \times 2 + \frac{n}{32} \times 3$$

$$\therefore \frac{1}{2}S = \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \dots = \frac{n}{2}$$

$$\therefore S = n \quad \therefore \text{时间复杂度为 } O(n)$$

down (x)

t=x

if (x*2 <= size && h[x*2] < h[t]) t=2x

if (x*2+1 <= size && h[x*2+1] < h[t]) t=2x+1

if (t != x)

swap (h[t], h[x])

down (t)

up (x)

if (x/2 >= 1 && h[x] < h[x/2])

swap (h[x], h[x/2])

up (x/2)

或 up (x)

while (x/2 >= 1 && h[x] < h[x/2])

swap (h[x], h[x/2])

x /= 2

如何修改或删除第 k 个插入的数

① ph[N] ② hp[N]

①: ph[k] 有的是第 k 个插入的点在堆中的下标

②: hp[k] 堆中下标为 k 的数是第几次插的

因为我们两个数交换时, 不仅要交换数本身

它们是第几次插入的性质也要变

h 中数据交换 ph、hp 也要交换

838. 堆排序

🏠 题目

📝 提交记录

💬 讨论

📖 题解

📺 视频讲解

输入一个长度为 n 的整数数列，从小到大输出前 m 小的数。

输入格式

第一行包含整数 n 和 m 。

第二行包含 n 个整数，表示整数数列。

输出格式

共一行，包含 m 个整数，表示整数数列中前 m 小的数。

数据范围

$1 \leq m \leq n \leq 10^5$,
 $1 \leq \text{数列中元素} \leq 10^9$

输入样例：

```
5 3
4 5 1 3 2
```

输出样例：

```
1 2 3
```

难度：

简单

时空限制：

1s / 64MB

总通过数：

102081

总尝试数：

157836

来源：

模板题

AcWing

算法标签

```
1 #include <stdio.h>
2
3 const int N = 1e5 + 10;
4 int n, m;
5 int hp[N]; // hp 0存储的是堆的大小
6
7 void down(int x) {
8     int t = x;
9     if(x * 2 <= hp[0] && hp[x * 2] < hp[t]) t = x * 2;
10    if(x * 2 + 1 <= hp[0] && hp[x * 2 + 1] < hp[t]) t = x * 2 + 1;
11    // 这里一定要注意， < hp[0]而不是n! 我们的堆的大小是在变化的!!
12    if(t != x) {
13        int tmp = hp[x]; hp[x] = hp[t]; hp[t] = tmp;
14        down(t);
15    }
16 }
17
18 int main() {
19     scanf("%d%d", &n, &m);
20     for(int i = 1; i <= n; i++) scanf("%d", &hp[i]);
21     hp[0] = n;
22     // 线性建堆
23     for(int i = n / 2; i >= 1; i--) down(i);
24
25     for(int i = 0; i < m; i++) {
26         printf("%d ", hp[1]);
27         hp[1] = hp[hp[0] --]; down(1);
28     }
29     return 0;
30 }
```

839. 模拟堆

题目

提交记录

讨论

题解

视频讲解

维护一个集合，初始时集合为空，支持如下几种操作：

1. `I x`，插入一个数 x ；

2. `PM`，输出当前集合中的最小值；

3. `DM`，删除当前集合中的最小值（数据保证此时的最小值唯一）；

4. `D k`，删除第 k 个插入的数；

5. `C k x`，修改第 k 个插入的数，将其变为 x ；

现在要进行 N 次操作，对于所有第 2 个操作，输出当前集合的最小值。

输入格式

第一行包含整数 N 。

接下来 N 行，每行包含一个操作指令，操作指令为 `I x`，`PM`，`DM`，`D k` 或 `C k x` 中的一种。

输出格式

对于每个输出指令 `PM`，输出一个结果，表示当前集合中的最小值。

每个结果占一行。

数据范围

$1 \leq N \leq 10^5$
 $-10^9 \leq x \leq 10^9$
数据保证合法。

输入样例：

```
8
I -10
PM
I -10
D 1
C 2 8
I 6
PM
DM
```

输出样例：

```
-10
6
```

难度：	简单
时/空限制：	1s / 64MB
总通过数：	63724
总尝试数：	163196
来源：	模板题
算法标签	

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 const int N = 1e5 + 10;
6 int n;
7 int h[N], ph[N], hp[N];
8 // ph[i] 第i次插入的数在堆中的下标
9 // hp[i] 堆中下标为i的数是第几次插入的
10 // h[0] 存储堆的大小
11
12 void heap_swap(int a, int b) {
13     // 注意，这里传入的是堆中的下标
14     int tmp;
15     tmp = ph[hp[a]]; ph[hp[a]] = ph[hp[b]]; ph[hp[b]] = tmp;
16     tmp = hp[a]; hp[a] = hp[b]; hp[b] = tmp;
17     tmp = h[a]; h[a] = h[b]; h[b] = tmp;
18     return ;
19 }
20
21 void down(int x) { // 下调操作
22     int t = x;
23     if(x * 2 <= h[0] && h[x * 2] < h[t]) t = x * 2;
24     if(x * 2 + 1 <= h[0] && h[x * 2 + 1] < h[t]) t = x * 2 + 1;
25     if(t != x) {
26         heap_swap(x, t);
27         down(t);
28     }
29     return ;
30 }
31
32 void up(int x) { // 上调操作
33     while(x / 2 >= 1 && h[x / 2] > h[x]) {
34         heap_swap(x, x / 2);
35         x /= 2;
36     }
37     return ;
38 }
39
40 void insert(int x, int k) { // 插入
41     h[++h[0]] = x;
42     ph[k] = h[0];
43     hp[h[0]] = k; // 把数放到堆尾，并更新相应的数据
44
45     up(h[0]); // 上调
46     return ;
47 }
48
49 int printfM() { // 返回最小值
50     return h[1];
51 }
52
53 void delM() { // 删除最小值
54     heap_swap(1, h[0]);
55     h[0]--; // 末尾数据覆盖堆顶
56
57     down(1); // 下调
58     return ;
59 }
60
61 void del(int k) { // 删除第k次插入的数
62     int pos = ph[k];
63     // 这里要注意，我们swap完之后 ph[k] = h[0]
64     // 我们必须记录删除前第k次插入的数在堆中的位置！！！！！！！！！！
65     heap_swap(h[0], ph[k]); // 末尾数据和要删除的数据交换
66     h[0]--;
67     down(pos); up(pos);
68     return ;
69 }
70
71 void change(int k, int x) { // 更改第k次插入的数据的值
72     h[ph[k]] = x;
73     down(ph[k]); up(ph[k]);
74     return ;
75 }
76
77
78
79
80 int main() {
81     scanf("%d", &n);
82     char choose[5]; int x, y;
83     int k = 0;
84     for(int i = 0; i < n; i++) {
85         scanf("%s", choose);
86
87         if(strcmp(choose, "I") == 0) {
88             scanf("%d", &x);
89             insert(x, ++k);
90         } else if(strcmp(choose, "PM") == 0) {
91             printf("%d\n", printfM());
92         } else if(strcmp(choose, "DM") == 0) {
93             delM();
94         } else if(strcmp(choose, "D") == 0) {
95             scanf("%d", &x);
96             del(x);
97         } else {
98             scanf("%d%d", &x, &y);
99             change(x, y);
100         }
101     }
102
103     return 0;
104 }
105 }
```

